citkid.pipeline user guide

`citkid.pipeline` calibrates and analyzes multitone KID data: target sweeps (fine and gain sweeps) and timestreams for many tones at once. It stores every result in one zarr store, computes what you ask for on demand, and deletes results that become stale when an earlier step is re-run.

This guide explains the concepts, the YAML files, custom steps, how invalidation works, and the interactive tools. `notebooks/pipeline.ipynb` walks through the same material with runnable cells, and `notebooks/ts_iq_analysis_minimal.ipynb`, `notebooks/sweep_series.ipynb`, `notebooks/iq_series.ipynb` and `notebooks/ts_series.ipynb` show complete sessions.

Quick start

```
import numpy as np
from citkid.pipeline import DataSet, AnalysisRunner, run_ts_analysis

DS = DataSet(
    zarr_path = 'path/to/ts_analysis.zarr',  # output store (created if new)
    cal_yaml_path = 'ts',                     # calibration YAML or alias
    custom_path = 'custom_steps_ts.py',       # your data-loading steps
)
AR = AnalysisRunner(
    DS,
    analysis_yaml_path = 'ts',                # analysis YAML or alias
    custom_path = 'custom_steps_ts.py',
)

on_res = np.where(DS.res_idxs[:] >= 0)[0]    # res_idxs < 0: calibration tones
AR.execute_path(data_idx = on_res)            # run every analysis step, save

run_ts_analysis(AR, data_idxs = on_res)       # review and refit row by row

DS.xt[on_res[0]]                              # calibrated timestream of a row
```

Later, possibly on another machine, reopen with the store path alone:

```
DS = DataSet(zarr_path = 'path/to/ts_analysis.zarr')
AR = AnalysisRunner(DS)
```

Concepts

Rows

A **row** is one tone (one resonator or calibration tone). Rows are indexed by `data_idx`, from 0 to `nrows - 1`. Every dataset must produce the global parameter `nrows` from a `'global'` calibration step (usually the first loading step). By convention `res_idxs` holds each tone's resonator index, with negative values for off-resonance calibration tones.

Parameters

Everything the pipeline handles is a named **parameter**: raw data (`ff`, `zf`, `zt`), intermediate products (`zf_rmv`, `thetat`), fit results (`p_amp`, `iq_popt`, `poly_x`) and user settings (`span_mult`, `iq_mask`). A parameter is either

- **global**: one value for the whole dataset (`nrows`, `fres_all`, `dt`, `fr_spans`). `DS.nrows` returns the value.
- **per-row**: one value per row (`ff`, `iq_popt`, `xt`). `DS.ff` returns a lazy accessor; index it to load rows:

  ```
  DS.ff[0]          # row 0
  DS.ff[[0, 1, 2]]  # rows 0-2, stacked (rows must have equal shapes)
  ```

```
DS.zt[0, :1000]    # first 1000 samples of row 0
DS.res_idxs[:]     # every row
```

Parameters are loaded lazily: from memory if present, else from the zarr store, else (for calibration parameters) computed by running the calibration steps that produce them.

Steps

A **step** (`plStep`) wraps one function with the names of its inputs and outputs. The pipeline fetches the inputs by name, calls the function, and stores the outputs by name. There are two kinds of step:

- **Calibration steps** (`DataSet`, calibration YAML) load the raw data and apply calibrations: remove the gain, center and rotate the IQ loop, convert to theta and x, compute PSDs. They run automatically whenever one of their outputs is requested, and their outputs are not saved unless you ask (`DS.write_params`). They have no user parameters.
- **Analysis steps** (`AnalysisRunner`, analysis YAML) fit things: the gain, the IQ circle, the x calibration, the IQ loop. They run only when you run them (`execute_path`, `execute_step`, or the GUI), their outputs are saved by default, and they can take user parameters from the YAML or the GUI.

The two are linked: calibration steps use analysis outputs (`rmv_gain_f` needs `p_amp` and `p_phase` from `fit_gain`; `get_xt` needs `poly_x` from `fit_x_theta`), and analysis steps use calibration outputs (`fit_iq_circle` needs `zf_rmv`). So `DS.xt[5]` works once the analysis has run for row 5: the dataset loads `zt`, removes the gain, centers, converts to theta, and applies `poly_x`.

Step types

The `func_type` of a step says how it is called:

`func_type`	Called	Inputs	Outputs
`'per-row'` (default)	Once per row	One row's values (global inputs as is)	One value per output for that row
`'vectorized'`	Once for all requested rows	Arrays with the requested rows on axis 0	Arrays with one entry per requested row on axis 0, even for one row
`'global'`	Once	Global parameters only	Global parameters
`'global-res'`	Once, for every row	Global parameters only	Per-row parameters: arrays of length `nrows` on axis 0

A per-row or vectorized step needs at least one per-row input (often the special input `data_idx`, which passes the row index or indices). If a value should differ between rows, make it per-row or vectorized, not global.

YAML files

A dataset has one calibration YAML and (optionally) one analysis YAML. Built-in templates are in `citkid/pipeline/templates/` and can be given by alias:

Alias	Calibration YAML	Analysis YAML	Use
`'ts'`	`cal_ts.yaml`	`analysis_ts.yaml`	On-resonance timestreams: gain, circle, x calibration, PSDs
`'ts_fixed_theta'`	`cal_ts.yaml`	`analysis_ts_fixed_theta.yaml`	As `'ts'`, but the x calibration mask is a fixed theta range
`'ts_offres'`	`cal_ts_offres.yaml`	`analysis_ts_offres.yaml`	Off-resonance timestreams (no circle fit)

Alias	Calibration YAML	Analysis YAML	Use
`'iq'`	`cal_iq.yaml`	`analysis_iq.yaml`	Target sweeps only: gain and nonlinear IQ fit

Anything that isn't an alias must be a path ending in `.yaml` or `.yml`.

Calibration YAML

The calibration YAML is a tree. This is `cal_ts.yaml`:

```yaml
CAL_STEPS:
  1 :
    task: load_global_data
  2 :
    task: load_global_res_data
    T_STEPS:
      1:
        task: load_ft
      2:
        task: load_zt
      3:
        task: rmv_gain_t
      4:
        task: center_t
      5:
        task: get_thetat
        A_STEPS:
          1:
            task: get_At
          2:
            task: get_sparper
          3:
            task: get_sfactor_reduced
      6:
        task: get_xt
        X_STEPS:
          1:
            task: get_sxx
          2:
            task: get_sxx_reduced
    F_STEPS:
      1:
        task: load_sweep_f # must be sorted on axis 1
      2:
        task: rmv_gain_f
      3:
        task: center_f
      4:
        task: get_thetaf
      5:
        task: get_xf
    G_STEPS:
      1:
        task: load_sweep_g
```

Rules (checked by `framework.check_pl_tree_structure`):

- Root keys and branch keys end in `_STEPS`. A `*_STEPS` value is a sequence with integer keys 1, 2, 3, ... with no gaps.
- Each sequence entry has a `task`: the name of a step (a custom step or a built-in one from `default_steps.default_cal_steps`).
- An entry can have its own `*_STEPS` branches, which run after it.
- Calibration entries can't have `params`.
- A `delete_input` key (removed; it never did anything) is ignored, so stores whose embedded YAML still has it can be reopened.

Order matters. To produce a parameter, the dataset runs the path to the step that outputs it: every step before it in its own sequence, every ancestor, and each ancestor's earlier siblings (but not their branches). For `xf` in the tree above the path is `load_global_data`, `load_global_res_data`, `load_sweep_f`, `rmv_gain_f`, `center_f`, `get_thetaf`, `get_xf`, even though `get_xf` itself only needs `ff` and `ft`. Inputs that aren't on the path (here `ft`) are produced through their own paths when the step asks for them. Put independent chains in separate `*_STEPS` branches so one doesn't wait on (or fail because of) the other. Use `framework.find_pl_path(DS.cal_pl, 'xf')` with `framework.print_path` to see a path.

Analysis YAML

The analysis YAML is a flat, ordered list under `ANALYSIS_STEPS` (the only allowed key), with optional `params` per step. This is `analysis_ts.yaml`:

```yaml
ANALYSIS_STEPS:
  1 :
    task: make_fr_spans
  2 :
    task: fit_gain
    params:
      span_mult: 1
  3 :
    task: fit_iq_circle
    params:
      circ_mask: null # null will build a mask that is all True.
  4 :
    task: get_idx_t
  5 :
    task: get_theta_phase_offset
  6 :
    task: get_xcal_mask
    params:
      xcal_idx0_offset: 3
      xcal_idx1_offset: 9
      xcal_std_cutoff: 16.0
  7 :
    task: fit_x_theta
```

- `params` values must be simple types: string, bool, int, float, or `null`. They become parameters of the dataset, as inputs of the step.
- `null` for one of the mask parameters `gain_mask`, `circ_mask`, `iq_mask` or `xcal_mask` becomes an all-True mask with the shape of one row of `fg` (for `gain_mask`) or `ff` (the others). Other `null` values are passed as `None`.
- User parameters of a per-row step are stored per row (each row keeps the value it was last run with, so the GUI can change them row by row). User parameters of a global step are stored as global parameters.
- Analysis steps run in the listed order. The step index (1-based in the YAML) is what invalidation uses: re-running step k invalidates the outputs of steps k+1, k+2, ...

Built-in steps

From `citkid/pipeline/default_steps.py`. Custom steps with the same name replace these.

Calibration steps:

Step	Function	Inputs	Outputs
`rmv_gain_f` / `rmv_gain_t`	`xcal.gain.remove_gain`	`ff`/`ft`, `zf`/`zt`, `p_amp`, `p_phase`	`zf_rmv` / `zt_rmv`
`center_f` / `center_t`	`xcal.circle.cent_rot_s21`	`zf_rmv`/`zt_rmv`, `circ_origin`, `theta_phase_offset`	`zf_cent` / `zt_cent`
`rotate_f_offres` / `rotate_t_offres`	`cent_rot_s21` with origin 0	`zf_rmv`/`zt_rmv`, `theta_phase_offset`	`zf_cent` / `zt_cent`
`get_thetaf`	`xcal.circle.convert_to_theta` (unwrapped at `idx_t`)	`zf_cent`, `idx_t`	`thetaf`
`get_thetat`	`convert_to_theta`	`zt_cent`	`thetat`
`get_xf`	`xcal.xcal.calc_x`	`ff`, `ft`	`xf`
`get_xt`	`np.polyval`	`poly_x`, `thetat`	`xt`
`get_At`	`xcal.circle.convert_to_A`	`zt_cent`	`At`
`get_sparper`	`xcal.circle.get_spar_sper`	`thetat`, `At`, `circ_radius`, `dt`	`f_sparper`, `spar`, `sper`
`get_sxx`	`signal.psd.get_psd`	`xt`, `dt`	`f_sxx`, `sxx`
`get_sxx_reduced`	`xcal.reduced_params.get_sxx_reduced`	`f_sxx`, `sxx`, default freqs	`sxx_0p1`, `sxx_0p3`, `sxx_1`, `sxx_3`, `sxx_10`, `sxx_30`, `sxx_100`, `sxx_300`
`get_sfactor_reduced`	`xcal.reduced_params.get_sfactor_reduced`	`f_sparper`, `sparper`, default freqs	`sfactor_0p1`, …, `sfactor_300`

All are per-row. The loading steps (`load_global_data`, `load_global_res_data`, `load_sweep_f`, `load_sweep_g`, `load_ft`, `load_zt`) are not built in: you provide them (see Custom steps).

Analysis steps:

Step	Type	Function	Inputs	Outputs
`make_fr_spans`	global	`xcal.gain.make_fr_spans`	`fres_all`, `qres_all`	`fr_spans`
`fit_gain`	per-row	`xcal.gain.fit_gain`	`fg`, `zg`, `fr_spans`, `span_mult`	`p_amp`, `p_phase`, `gain_mask`
`fit_iq_circle`	per-row	`xcal.circle.fit_iq_circle`	`zf_rmv`, `circ_mask`	`circ_origin`, `circ_radius`
`get_idx_t`	per-row	`xcal.xcal.nearest_index`	`ft`	`idx_t`
`get_theta_phase_offset`	per-row	`xcal.circle.get_theta_phase_offset`	`zt_rmv`, `circ_origin`	`theta_phase_offset`
`get_theta_phase_offset_offres`	per-row	`get_theta_phase_offset` with origin 0	`zt_rmv`	`theta_phase_offset`
`get_xcal_mask`	per-row	`xcal.xcal.get_xcal_mask`	`fg`, `thetaf`, `thetat`, `xcal_idx0_offset`, `xcal_idx1_offset`, `xcal_std_cutoff`	`xcal_mask`
`get_xcal_mask_fixed_theta`	per-row	`xcal.xcal.get_xcal_mask_fixed_theta`	`fg`, `thetaf`, `xcal_theta_min_offset`, `xcal_theta_max_offset`	`xcal_mask`
`fit_x_theta`	per-row	`xcal.xcal.fit_x_theta`, degree 3	`thetaf`, `xf`, `xcal_mask`	`poly_x`

Step	Type	Function	Inputs	Outputs
`fit_iq`	per-row	`res.fitter.fit_nonlinear_iq` (default guess and bounds, downward sweep)	`freq_mv`, `iq_mask`	`iq_p0`, `iq_popt`, `iq_nrmse`

`fit_x_theta` always fits a degree-3 polynomial, on purpose: `poly_x` is stored as one array per dataset, so every row must have the same shape. For another degree, write a custom analysis step (with a new output name if you mix degrees across datasets).

Custom steps

You write the steps that load your raw data, and optionally extra calibration or analysis steps, in a Python file. Start from `templates/custom_steps_ts.py` (timestreams) or `templates/custom_steps_iq.py` (target sweeps only). The file must define `custom_cal_steps` (a list of `plStep`), and may define `custom_analysis_steps`:

```
import os
import numpy as np
import zarr
from citkid.pipeline.framework import plStep

# main_dir must be the only hard-coded path. It can be replaced with
# DataSet(..., custom_main_dir_overwrite=...) after the data is moved.
main_dir = '/path/to/data/'
data_path = os.path.join(main_dir, 'example.zarr')
root = zarr.open(data_path, mode = 'r')

def load_global_data():
    fres_all = root['fres_all'][...]
    qres_all = np.ones_like(fres_all) * 6000
    nrows = root['fres'].shape[0]
    return fres_all, qres_all, nrows

def load_global_res_data():
    return (root['fres'][...], root['qres'][...], root['ares'][...],
            root['res_idxs'][...])

def load_sweep_f(data_idx):
    ff = np.array(root['f'][data_idx, :])
    zf = np.array(root['z'][data_idx, :])
    idx = np.argsort(ff)              # sweeps must be sorted by frequency
    return ff[idx], zf[idx]

def load_sweep_g(data_idx):
    fg = np.array(root['f'][data_idx, :])
    zg = np.array(root['z'][data_idx, :])
    idx = np.argsort(fg)
    return fg[idx], zg[idx]

custom_cal_steps = [
    plStep('load_global_data', load_global_data,
           [], ['fres_all', 'qres_all', 'nrows'], 'global'),
    plStep('load_global_res_data', load_global_res_data,
           [], ['fres', 'qres', 'ares', 'res_idxs'], 'global-res'),
```

```
    plStep('load_sweep_f', load_sweep_f, ['data_idx'], ['ff', 'zf'], 'per-row'),
    plStep('load_sweep_g', load_sweep_g, ['data_idx'], ['fg', 'zg'], 'per-row'),
]
custom_analysis_steps = []
```

`plStep(name, func, param_names, return_names, func_type = 'per-row'):`

- `param_names` and `return_names` are lists of parameter names, in the function's argument and return order. A function with several outputs returns a tuple; a dict is not unpacked.
- `data_idx` as an input passes the row index (an int for a per-row step, an array of rows for a vectorized one).
- A step in the file overrides the built-in step of the same name.
- Output names can't be the names of `DataSet` methods (`plot`, `copy`, `apply_cal`, ...). Also avoid the names of `DataSet` attributes such as `root`, `cal_steps` or `zarr_path`, which would hide the parameter.
- The timestream template adds `load_ft` (tone frequency `ft`) and `load_zt` (timestream `zt`, scaled by `counts_to_s21`) as vectorized steps, and `dt` to `load_global_data`.

Add your steps to the YAML (as `task: <name>`), and give the file to both `DataSet(custom_path = ...)` and `AnalysisRunner(custom_path = ...)` (the dataset reads `custom_cal_steps`, the runner reads `custom_analysis_steps`).

Check the steps with `DS.check_cal(data_idx)`. It runs every calibration step for that row without caching or saving, and prints each step's output types, shapes and dtypes, structure problems (a global-res output without one entry per row, `nrows` not a whole number, unsorted frequencies, `ff` and `zf` of different lengths), and for a failing step the error with a traceback into your function. Steps that need analysis outputs the dataset doesn't have yet are reported as needing analysis.

Other ways to give calibration steps:

- `DataSet(custom_source = ...)`: the source code as a string, embedded like a file. `pipeline.sweep_series.target_sweep_source` builds such source for data saved by `crs.procedures.target_sweep` / `power_sweep`, and `sweep_series.open_series_datasets` uses it to open one `DataSet` per set, so series of target sweeps need no custom file. See `notebooks/sweep_series.ipynb`.
- `DataSet(custom_cal_steps = [...])`: a list of `plSteps`. These are not embedded, so the dataset can't be reopened without them; this needs `cal_yaml_path` every time.

Running the analysis

```
AR = AnalysisRunner(DS, analysis_yaml_path = 'iq')

AR.execute_path()                                # every step, every row
AR.execute_path(data_idx = [0, 1, 2], step_start_idx = 1)
AR.execute_path(step_start_idx = 2, step_end_idx = 4)
```

`execute_path(data_idx = None, step_start_idx = 0, step_end_idx = None, verbose = True, save = True, vectorize = True, path_per_row = False):`

- `data_idx`: rows to process (None: every row). Global steps ignore it.
- `step_start_idx`, `step_end_idx`: zero-based indices into `AR.path` (the YAML steps in order), like a slice. They count steps, not rows.
- `save`: save each step's user parameters and outputs when it finishes (default True).
- `vectorize = False` runs vectorized steps one row at a time (less memory; a failing row is recorded instead of failing the call).
- `path_per_row = True` runs the whole remaining path for one row before the next (implies `vectorize = False`). Global steps still run once.

A single step:

```
fit_iq_step = next(s['task'] for s in AR.path if s['task'].name == 'fit_iq')

mask = np.ones(DS.ff[5].shape, dtype = bool)
mask[:10] = False
AR.execute_step(fit_iq_step, data_idx = [5], user_params = {'iq_mask': mask})
```

```
AR.execute_step(fit_iq_step, data_idx = [7], user_params = {'iq_mask': None},
                save = False)                    # keep in memory for now
AR.save_step_outputs(fit_iq_step, data_idx = [7])
```

`execute_step(step, data_idx = None, user_params = 'from_yaml', save = True, vectorize = True, allow_global_step_overwrite = False)`:

- `user_params = 'from_yaml'` (default) uses the step's `params` from the analysis YAML. A dict replaces them (parameters it leaves out are not taken from the YAML). For an array parameter on several rows, give one entry per row (shape `(len(data_idx), ...)`).
- It runs only the step you ask for. If an input comes from an earlier analysis step that hasn't run for these rows, it raises and names that step. Calibration inputs are computed as needed.
- `save_step_outputs(step, data_idx = None)` saves the latest results of a step that ran with `save = False`, without re-running it.

Failures. If a per-row step raises for some rows, those rows are recorded (with tracebacks) in `AR._last_failures`, in the zarr group `_failures/<step name>`, and in a `RuntimeWarning`; the other rows carry on. `execute_path` leaves failed rows out of the later steps, and raises `RuntimeError` (with the first traceback) only if every requested row failed. A vectorized step run with `vectorize = True`, or a global step, raises on failure.

Other tools:

- `DS.plot(data_idx, plot_type)` and `DS.plot_full_cal(data_idx)`: quick matplotlib plots. `plot_type` is one of `'raw_data'`, `'gain_fit'`, `'s21_rmv'`, `'circfit'`, `'sparper'`, `'xcal'`, `'iq_fit'`.
- `DS.write_params(names, data_idx = None)`: save parameters (computing calibration parameters if needed), e.g. `DS.write_params(['xt'])` to store the calibrated timestreams.
- `DS.apply_cal(data_indices, outputs, replacements = None)`: compute calibration outputs with temporary inputs, without changing the dataset, e.g. `DS.apply_cal([3], 'xt', {'zt': my_zt})`.
- `AR.release_rows(data_idx)` (or `DS.release_rows`): drop finished rows from memory in long loops.
- `DS.saved_row_mask(name)`: which rows of a parameter are saved.
- `DS.clear_cache()`: forget everything in memory and reread the store.

Invalidation

The pipeline keeps one value per parameter and row, and never lets a result outlive what it was computed from. When you run a step (from code or the GUI), for the rows it runs on (all rows for a global or global-res step) it

1. removes from memory the step's own outputs, the outputs of every later step in the analysis path, and every calibration parameter computed from any of those or from the step's user parameters (for example `zf_rmv`, `thetaf`, `xt` and `sxx` after `fit_gain`);
2. runs the step and stores its new user parameters and outputs;
3. if it saves (the default), deletes the same downstream parameters from the zarr store for those rows and writes the new inputs and outputs.

So changing a user parameter and re-running is the same as any re-run: later results for those rows are gone until you run the later steps again, and the calibration recomputes from the new fit when you next access it.

Details:

- "Later" means later in the analysis YAML, whether or not the later step actually uses the outputs. A step that isn't in the analysis path (such as `fit_iq` in the `run_iq_ts_analysis` window) isn't invalidated by the steps in the path.
- Re-running a calibration step through `execute_step` invalidates the whole analysis path for those rows.
- With `save = False`, only memory is invalidated. The old values stay in the store, hidden for the rest of the session, until you save (`save_step_outputs`). If you never save, a later session sees the old, consistent set of saved values.
- Re-running a global or global-res step whose outputs, or anything after it, already exist would invalidate every row. `execute_step` refuses unless you pass `allow_global_step_overwrite = True`. `execute_path` refuses too (see Common errors): start after the global step with `step_start_idx`.

Storage

Zarr layout

```
output.zarr/
    zarr.json                root attrs['pipeline']: cal_yaml, cal_custom_source,
                             analysis_yaml, analysis_custom_source, schema_version
    fr_spans/data            global parameter: one array
    iq_popt/data             per-row parameter: shape (nrows, ...), one chunk per row,
    iq_popt/row_exists       sharded in blocks of 256 rows; bool mask of saved rows
    iq_popt/pending_data     only with write_buffer=True, until consolidated
    iq_popt/pending_row_exists
    _failures/<step>         attrs['failures']: {'idx<data_idx>': error, traceback, time}
```

Each parameter group has attributes `global`, `pipeline_scope` (`'cal'` or `'analysis'`), `step_name`, `step_index` and `write_time`. Rows that were never saved hold the fill value; check `row_exists` (or `DS.saved_row_mask`) when reading the arrays directly. Calibration parameters are only in the store if you saved them with `write_params`. Entries in `_failures` stay after the row later succeeds.

Embedding and reopening

The first time a store is used, `DataSet` writes the contents (not the paths) of the calibration YAML and the custom calibration source into the store's metadata, and `AnalysisRunner` does the same for the analysis YAML and custom analysis source. Built-in alias templates are embedded too. You can then reopen with only the store, even after the YAML and step files have moved or changed:

```
DS = DataSet(zarr_path = 'path/to/output.zarr')
AR = AnalysisRunner(DS)
```

If you pass a YAML or custom file again, it must match what is embedded (custom source is compared ignoring the `main_dir` value). Otherwise you get

```
ValueError: Provided calibration YAML does not match the dataset definition
ValueError: Provided calibration custom steps do not match the dataset definition
ValueError: Provided analysis YAML does not match the dataset definition
ValueError: Provided analysis custom steps do not match the dataset definition
```

This protects a store from mixing results of different definitions. To change the definition, write to a new store (or delete the old one). Steps passed with `custom_cal_steps` are not embedded and not checked.

Moving the raw data

The only hard-coded path in a custom step file should be a top-level, unindented `main_dir = ...` assignment, with every other path built from it. When the raw data moves, pass the new directory each time you open the store:

```
DS = DataSet(zarr_path = 'path/to/output.zarr',
             custom_main_dir_overwrite = r'new/path/to/raw/data')
AR = AnalysisRunner(DS)    # uses DS.custom_main_dir_overwrite too
```

The first top-level `main_dir = ...` line in the embedded calibration and analysis source is replaced before the source runs. The embedded source itself is not changed. A warning is shown if the source has no such line.

Write buffer

`DataSet(..., write_buffer = True)` writes per-row results to a fast unsharded buffer (one file per row) instead of rewriting shards. Reads through a `DataSet` include buffered rows, but tools that open the zarr arrays directly don't see them. Call `DS.consolidate_storage()` when done (and before copying the store) to merge them into the shards. The series windows do this when they close.

Threads

A `DataSet` is not meant to be shared across threads. `DS.copy()` opens another `DataSet` on the same store and definition, with its own caches, for a worker thread. Writes to one store are serialized by a shared lock.

Interactive review

The GUIs take an `AnalysisRunner` and step through rows. The Qt event loop blocks until the window closes.

```
from citkid.pipeline import (run_iq_analysis, run_ts_analysis,
                             run_iq_ts_analysis, run_gain_only_analysis)

run_ts_analysis(AR, start_idx = 0, data_idxs = on_res,
                ui_scale = 1.0, plot_scale = 1.0)
```

Launcher	Panels	For
`run_iq_analysis`	gain fit, IQ fit	`'iq'`
`run_ts_analysis`	gain fit, circle fit, x calibration	`'ts'`, `'ts_fixed_theta'`
`run_iq_ts_analysis`	the `run_ts_analysis` panels, then IQ fit	`'ts'` plus an IQ fit
`run_gain_only_analysis`	gain fit	e.g. `'ts_offres'`
`run_interactive(AR, panels = None, ...)`	one panel per step of `AR.path`, or your grouping	any analysis

`start_idx` is a position in `data_idxs` (not a `data_idx`); `data_idxs` is the list of rows to step through (default: every row). `ui_scale` scales text and widgets, `plot_scale` the plot heights.

Each panel runs one or more steps for the current row:

Panel	Steps	Settings
`GainFitPanel`	`fit_gain`	`span_mult`
`CircleFitPanel`	`fit_iq_circle`, `get_idx_t`, `get_theta_phase_offset`	`circ_mask` (drag on the plot)
`XCalPanel`	`get_xcal_mask`, `fit_x_theta`	the `get_xcal_mask` parameters
`XCalFixedThetaPanel`	`get_xcal_mask_fixed_theta`, `fit_x_theta`	`xcal_theta_min_offset`, `xcal_theta_max_offset`
`IQFitPanel`	`fit_iq`	`iq_mask` (drag on the plot)

Steps without a registered panel get a `DefaultStepPanel`.

- **Run** runs the panel with its settings, **Run+** runs it and every panel after it. Re-running a panel invalidates the later panels for that row and clears their plots; the window asks before saving or moving on while later panels still need a run.
- **Save** saves the panel's results; they are also saved when you move to another row. **Bad Data** stores NaN results for the row.
- Keys: A/D or Left/Right (previous/next row), R (rescale plots), 1-9 (Run+ on panel N), Shift+1-9 (run panel N only), H (help).
- Global analysis steps (such as `make_fr_spans`) run automatically when a panel needs their outputs and they don't exist yet.

The window doesn't run steps that have no panel. With the `'ts_offres'` analysis, for example, `run_gain_only_analysis` fits the gain; run `get_idx_t` and `get_theta_phase_offset_offres` afterwards with `AR.execute_path(step_start_idx = 2)`.

To add a panel for a custom step, subclass `citkid.pipeline.interactive.StepPanel`, override `setup_ui`, `get_params_for_step` and `update_plots`, and register it with `@register_panel('<step name>', ...)`. See `interactive/gain.py` for a small example.

Series

For a series of measurements of the same tones (tone power, blackbody temperature, bath temperature, ...), each series point has its own `DataSet`:

- `pipeline.sweep_series.open_series_datasets(main_dir, data_path, out_root, ...)` opens one `DataSet` per `set{i}` group of a `power_sweep`-style store, and `run_series_analysis(datasets, ...)` runs the analysis on all of them.
- `run_iq_series(datasets = ..., root = ..., x_param_name = 'ares', y_param_name = 'a', xy_fit = ...)` shows a fit parameter of every series point vs the series parameter, for each resonator, next to the gain and IQ fit panels of the selected point, and fits the rows in the background. `run_ts_series` does the same with the `'ts'` panels and any scalar output (e.g. `sxx_10`) on x and y.
- `sweep_series.a_vs_power_fit` and `responsivity_fit` are ready-made `SeriesXYFits`; `pick_optimal_ares` and `save_popt` pick and save each resonator's optimal tone power.
- With `y_target` (e.g. the nonlinearity parameter a_{nl} to tune to), `run_iq_series` marks the x at which each resonator's fit reaches the target with a vertical line; Shift+click the plot (or drag the line) to choose a different x for that resonator, and Reset to return to the fit's. The chosen x of every resonator is saved as `root['xy_fit/x_chosen']` (`x_chosen_user` marks the ones chosen by hand).

Both windows also accept `make_custom_steps` (a function of the series index returning loading steps) with `root` and `n_series` instead of `datasets`. See `notebooks/sweep_series.ipynb`, `notebooks/iq_series.ipynb` and `notebooks/ts_series.ipynb`.

Limitations of the templates

- `fit_x_theta` is fixed at degree 3 (see Built-in steps).

Common errors

`Re-running global step 'make_fr_spans' would overwrite its existing outputs or downstream products for all rows.` You ran `execute_path()` again on a dataset that already has results. Start after the global step: `AR.execute_path(step_start_idx = 1)`. To really recompute `fr_spans` (and invalidate every row), run that step on its own with `allow_global_step_overwrite = True`, then the rest of the path with `step_start_idx = 1`. (`execute_path` itself has no `allow_global_step_overwrite` option.)

`Partial execute_path would execute global step 'make_fr_spans', ... rerun with step_start_idx=1.` Same cause, with a subset of rows: a global step can't be re-run for only some rows. Use the suggested `step_start_idx`.

`Parameter 'x' is produced by earlier analysis step 'y'. Run that step explicitly, ...` `execute_step` needs an output of an earlier analysis step that doesn't exist for these rows. Run the earlier steps first (or `execute_path` from that step).

`Provided calibration YAML does not match the dataset definition.` (or the analysis YAML, or custom steps). A dataset stores its YAML and custom steps and compares them, as text, with any `cal_yaml_path`, `analysis_yaml_path` or `custom_path` you pass. If the citkid templates or your custom file changed since the dataset was created, they no longer match. To reopen an existing dataset, pass only `zarr_path` (and `custom_main_dir_overwrite` if the data moved), so the stored definition is used.

`Step 'rotate_offres_t' has been renamed to 'rotate_t_offres'.` The dataset was created before citkid renamed some built-in steps (listed in `citkid.pipeline.migrate.RENAMED_STEPS`). Update it once, while nothing else has it open:

```
from citkid.pipeline import migrate_step_names
migrate_step_names(zarr_path, dry_run = True)  # prints the planned changes
migrate_step_names(zarr_path)
```

This renames the steps in the stored YAML, the `step_name` labels of saved parameters and the `_failures` records, and keeps the original YAML in the dataset metadata (`before_step_rename_cal_yaml`). Migrated datasets can't be opened by citkid versions from before the rename. If the old name comes from your own YAML file, rename the task there instead.

`Step 'load_data_f' has been renamed to 'load_sweep_f' in the templates.` The template loaders `load_data_f/_g` are now `load_sweep_f/_g`; use the new names in your YAML. (Old datasets don't need updating for these: they store their own custom steps, which still define the old names.)

`Step 'load_sweep_f' not found, but the custom steps define 'load_data_f', its old name.` Your custom-steps file was copied from an older template. Rename the step (and its function) in that file to the new name to use it with the current templates.

`Step 'x' requires parameter 'y', but it is not available.` No step produces `y` and it isn't stored. Check the spelling in `plStep`, and that the step producing it is in the YAML.

`Parameter 'ff' is missing rows [0] and cannot be produced, because calibration step 'load_sweep_f' failed for data_idx 0: ....` A calibration step raised. The message includes the traceback into your function and a summary of its inputs (type, shape, dtype). Run `DS.check_cal(0)` for a full report.

`Step 'x' returned 1 output(s) but has 2 return names.` The function must return a tuple in `return_names` order (a dict counts as one output).

`Vectorized step 'x' was called for 3 row(s) but output 'y' is ...` or **`Global-res step 'x' output 'y' is ..., but a global-res step must return one entry per resonator.`** Vectorized and global-res outputs need the rows on axis 0, even for a single row.

`Calibration pipeline must be able to produce 'nrows' as a global parameter.` Add `nrows` to the outputs of a `'global'` loading step.

`Step 'x' not found in available steps.` A `task` in a YAML names a step that is neither built in nor in your custom file (check that the file is passed, and to the right class: calibration steps to `DataSet`, analysis steps to `AnalysisRunner`).

`... does not match the dataset definition.` See Embedding and reopening.

`No calibration YAML was provided and no embedded definition exists in the dataset.` A new store needs `cal_yaml_path`.

`Every requested row failed. First failure: step 'x' for data_idx n: ....` Every row failed in some step; the traceback of the first is included. Fix that step (often a loading step) and run again.